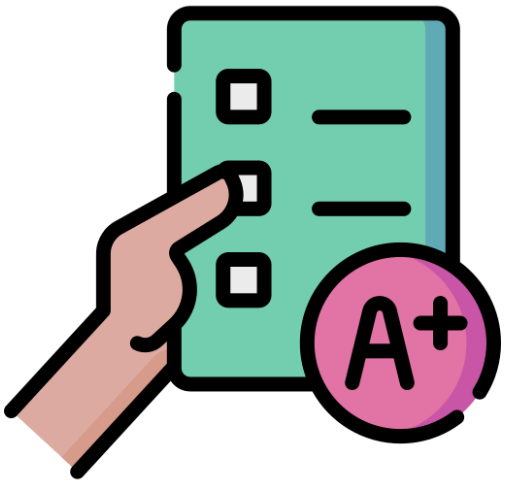

Programación I

Sistema de Calificaciones UADE

Grupo 1

Docente: Abel Israel Laime Huanca

Alumnos: Sofia Fernandez, Jesus Espinoza y Barbara Coutinho

Índice

Programación I 1
Índice 2
- **Introducción** 3
- **Proyecto y contexto** 4
- **Objetivos** 5
- **Requisitos del proyecto** 6
- **Equipo de trabajo** 7
- **Primeros bocetos** 8
- **Explicación de la lógica del código** 12
- **Estructura general del proyecto (directorios)** 13
- **Listas y cadenas en uso** 14
- **Capturas de pantalla** 15
- **Futuras actualizaciones** 17

Introducción

En el ambiente educativo, la correcta administración y almacenamiento de las notas es importante, ya que permite evaluar el rendimiento académico de los estudiantes y facilitar la toma de decisiones pedagógicas. Este proyecto propone el desarrollo de un **sistema de calificaciones**, implementado en el lenguaje de programación **Python**, que permite registrar estudiantes, agregar materias, cargar sus notas y calcular promedios utilizando estructuras de datos.

Proyecto y contexto

El sistema está orientado al registro y consulta de información académica en un entorno simple y didáctico.

Surge en el marco de la práctica universitaria de programación estructurada, con el objetivo de reforzar conceptos básicos de programación y la aplicación de estructuras de datos.

Si bien el alcance inicial es académico y de uso limitado, este proyecto puede servir como base para el desarrollo de sistemas más complejos, con interfaces gráficas o integración a bases de datos.

Repositorio Github

https://github.com/jesuse99/proyecto_final_prog1

Objetivos

Objetivo General

Desarrollar un sistema en consola que permita gestionar estudiantes y sus calificaciones de manera organizada, utilizando Python como herramienta principal.

Objetivos Específicos

- Diseñar un menú interactivo que facilite la navegación por las distintas funcionalidades.
- Permitir el registro, actualización y eliminación de estudiantes con identificadores únicos.
- Incorporar el ingreso de calificaciones en formato de matriz asociada a cada estudiante.
- Permitir el registro, actualización y eliminación de materias,
- Implementar funciones que calculen el promedio individual de cada estudiante y el promedio general por materia.
- Implementar validaciones para cada uno de los tipos de datos.
- Favorecer la comprensión y aplicación de diccionarios y listas en Python.
- Almacenamiento y persistencia de datos en archivo CSV

Requisitos del proyecto

Funcionalidad	Asignado	Desarrollado	Testeado
Mostrar estudiantes	✅	✅	✅
Mostrar materias	✅	✅	✅
Mostrar notas	✅	✅	✅
Mostrar carreras	✅	✅	✅
Agregar estudiantes	✅	✅	✅
Agregar materias	✅	✅	✅
Agregar notas	✅	✅	✅
Agregar carreras	✅	✅	✅
Editar estudiantes	✅	✅	✅
Editar notas	✅	✅	✅
Calcular promedio por materia	✅	✅	✅
Buscar estudiante por legajo	✅	✅	✅
Buscar notas por legajo	✅	✅	✅
Buscar estudiantes por carrera	✅	✅	✅
Agregar validaciones	✅	✅	✅
Persistencia de datos	✅	✅	✅

Equipo de trabajo

El proyecto se desarrollará en equipo, con la siguiente asignación de responsabilidades:

- Sofia Fernandez (Team Leader) : desarrollo del sistema, testing y documentación.
- Jesus Espinoza (Proyect Manager): desarrollo del sistema, testing y documentación.
- Barbara Coutinho (Testing): desarrollo del sistema, testing y documentación.

Primeros bocetos

Menú principal:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Opción: __
```

Opción 1: Ingresar datos

Ingreso:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción: 1

=== INGRESAR DATOS ===

1. Agregar estudiantes
2. Agregar materias
3. Salir

Seleccione una opción: _
```

Opción 2: Editar datos

Ingreso:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción: 2

=== EDITAR DATOS ===

1. Editar estudiantes
2. Editar notas
3. Salir


Seleccione una opción: _
```

Opción 3: Gestionar promedios

Ingreso:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción: 3

=== GESTIONAR PROMEDIOS ===
1. Calcular promedio de notas de un estudiante
en una materia
2. Volver
```

```
Seleccione una opción: _
```

Opción 4: Buscar por criterio

Ingreso:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción: 4

=== BUSCAR POR CRITERIO ===
   1. Buscar estudiante por legajo
   2. Buscar notas por legajo de estudiante
   3. Buscar estudiantes por carrera
   4. Volver

Seleccione una opción: _
```

Opción 5: Mostrar datos

Ingreso:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción: 5

=== MENÚ MOSTRAR DATOS ===
   1. Mostrar Alumnos
   2. Mostrar Notas
   3. Volver

Seleccione una opción: _
```

Explicación de la lógica del código

El programa implementa un sistema de gestión de calificaciones con un menú interactivo que permite:

1. **Ingresar datos:**
   - Añadir estudiantes, materias, carreras y notas.
2. **Editar datos:**
   - Modificar estudiantes, carreras y notas existentes.
3. **Gestionar promedios:**
   - Calcular el promedio de notas de un estudiante en una materia.
4. **Buscar por criterio:**
   - Consultar información según legajo, materia o carrera.
5. **Mostrar datos**
   - Listar estudiantes, materias, carreras y notas registradas.
6. **Salir**
   - Cerrar el programa.

La lógica sigue el patrón:

Menú principal → Submenú → Acción → Validaciones → Resultado → Retorno al menú.

Las listas utilizadas simulan una base de datos y son compartidas entre funciones para mantener la persistencia en memoria durante la ejecución.

Estructura general del proyecto (directorios)

El proyecto está organizado de forma modular bajo el enfoque de **programación estructurada**, dividiendo la lógica en distintos archivos para mejorar la claridad y el mantenimiento.

/Proyecto_Sistema_Calificaciones

├── main.py

├── **functions**/

│ ├── menu.py

│ ├── auxiliars.py

│ |── validations.py

│ ├── students.py

│ |── subjects.py

│ ├── careers.py

│ |── notes.py

├── **utils**/

│ ├── data.py

│ ├── service.py

Descripción de los módulos:

- **main.py** → Es el punto de entrada del programa. Gestiona el flujo principal mediante un menú interactivo, llamando todas las funciones de menu.py..

- **menu.py** → Invoca a las funciones principales y presenta los menús de navegación que implementan los módulos de la aplicación:(students, subjects, careers, notes)

- **auxiliars.py** → Incluye funciones auxiliares o de soporte (por ejemplo, para calcular promedios, generar identificadores aleatorios, o filtrar datos).

- **validations.py** → Centraliza todas las funciones de validación (entradas de usuario, existencia de datos, formato de texto y números).

- **students.py** → Contiene todas las funciones y validaciones relacionadas con estudiantes. (agregar, editar, buscar, etc)

- **subjects.py** → Contiene todas las funciones y validaciones relacionados con materias (agregar, buscar, calcular promedio, etc)

- **careers.py** → Contiene todas las funciones y validaciones relacionados con carreras (mostrar todas, agregar, etc.)

- **notes.py** → Contiene todas las funciones y validaciones relacionados con notas (mostrar todas, agregar, editar, etc.)

- **data.py** → Posee todas las estructuras de datos, con sus correspondientes datos que necesita el programa para funcionar.

- **service.py** → Gestiona la persistencia de datos del programa, permite leer los datos del archivo CSV y escribir en el mismo, luego de un cambio o actualización..

Persistencia de datos

Observamos que al cerrar el programa, luego de agregar o editar datos dentro del mismo, se pierden todos los cambios realizados. Por lo que, para solucionar este inconveniente, se agregó e implementó persistencia de datos en un archivo, formato CSV, el cual permite almacenar la información y poder recuperarla sin pérdida.

Para esto, se crearon dos funciones principales dentro del archivo **service.py** .

La primera permite obtener los datos del archivo, para esto recibe como parámetro el nombre del archivo del cual se quiere obtener los datos. Dentro de la función, se abre el archivo, en modo lectura, se recorren todos las filas y se guarda en una variable. Una vez leída todas las líneas (excepto el encabezado) del archivo y copiadas a la variable de tipo lista, se finaliza la función y se devuelve la variable que contiene todos los datos.

Todo esto dentro de una estructura try-except que permite evitar cualquier error o cierre inesperado del programa al leer el archivo

La segunda función, permite guardar los datos en el archivo, para lo cual recibe como parámetro el nombre del archivo y los datos a guardar en el mismo. En primer lugar se abre al archivo, ahora en modo escritura y se agregan todos los datos recibidos como parámetro. Para solucionar el inconveniente, que se pierde el encabezado al escribir en el archivo, se agregó una función **get_head()** que devuelve el encabezado del archivo según el nombre del mismo.

Al igual que la anterior función toda la lógica dentro de una estructura try-except que permite evitar cualquier error o cierre inesperado del programa al escribir en archivo

Listas y cadenas en uso

El programa utiliza listas de listas para representar las entidades principales:

- Students: [id, legajo_estudiante, nombre, codigo_carrera]
- Subjects: [id, codigo_materia, nombre, codigo_carrera]
- Careers: [id, codigo_carrera, nombre, facultad]
- Notes: [id, codigo_materia, legajo_estudiante, valor_nota]

Estas listas permiten recorrer los datos con bucles y realizar búsquedas mediante comparaciones de cadenas o números.

```
# careers = []
careers = [
    [1, "1010", "Ingeniería en Sistemas", "Facultad de Ingeniería"],
    [2, "2020", "Medicina", "Facultad de Ciencias Médicas"],
    [3, "3030", "Derecho", "Facultad de Ciencias Jurídicas"],
    [4, "4040", "Ciencias Económicas", "Facultad de Ciencias Económicas"],
    [5, "5050", "Psicología", "Facultad de Humanidades"],
    [6, "6060", "Arquitectura", "Facultad de Arquitectura"]
]
```

```
# students = []
students = [
    [1, "1111", "Sofía Fernandez", "1010"],
    [2, "2222", "Marcos Garcia", "2020"],
    [3, "3333", "Lucía Lopez", "6060"],
    [4, "4444", "Ana Diaz", "3030"]        #
]
```

```
# subjects = []
subjects = [
    [1, "1532", "Programación I", "1010"],     #
    [2, "2897", "Bases de Datos", "1010"],     #
    [3, "4120", "Anatomía", "2020"],           #
    [4, "5784", "Derecho Penal", "3030"],      #
    [5, "6962", "Macroeconomía", "4040"],      #
    [6, "8459", "Psicología Cognitiva", "5050"] #
]
```

```
# notes = []
notes = [
    [1, "1532", "1111", 90],
    [2, "2897", "1111", 80],
    [3, "4120", "2222", 70],
    [4, "5784", "4444", 60],
]
```

Capturas de pantalla

Menú principal:

```
=== SISTEMA DE CALIFICACIONES UADE ===

1. Ingresar datos
2. Editar datos
3. Gestionar promedios
4. Buscar por criterio
5. Mostrar datos
6. Salir

Seleccione una opción:
```

Opción 1. Ingresar datos

```
=== INGRESAR DATOS ===

1. Agregar estudiantes
2. Agregar materias
3. Agregar notas
4. Agregar carreras
5. Volver

Seleccione una opción:
```

Opción 2. Editar datos

```
=== EDITAR DATOS ===

1. Editar estudiantes
2. Editar notas
3. Volver

Seleccione una opción: 3
```

Opción 3. Gestionar promedios

```
=== GESTIONAR PROMEDIOS ===

    1. Calcular promedio de notas de un estudiante en una materia
    2. Volver

Seleccione una opción:
```

Opción 4. Buscar por criterio

```
=== BUSCAR POR CRITERIO ===

    1. Buscar estudiante por legajo
    2. Buscar notas por legajo de estudiante
    3. Buscar estudiantes por carrera
    4. Volver

Seleccione una opción:
```

Opción 5. Mostrar datos

```
 === MENÚ MOSTRAR DATOS ===

 1. Mostrar estudiantes
 2. Mostrar notas
 3. Mostrar materias
 4. Mostrar carreras
 5. Volver
```

Opción 6. Salir

```
6. Salir

Seleccione una opción: 6
Cerrando programa..
Programa finalizado
```

Futuras actualizaciones

Algunas mejoras y ampliaciones que podrían implementarse son:

1. **Uso de diccionarios para organizar la información:** Sustituir las listas por diccionarios o listas de diccionarios, lo que permitiría acceder a los datos por clave (por ejemplo, student["name"]), haciendo el código más legible y eficiente.
2. **Persistencia de datos:** Guardar la información en archivos (JSON, CSV) para mantener los registros después de cerrar el programa. ✓ **(entrega 13/10)**
3. **Gestión de fechas y estadísticas:** Agregar campos de fecha de examen y calcular estadísticas globales (promedios por materia, ranking, etc.). ✓ **(entrega 13/10)**
4. **Eliminación de registros:** Incorporar la funcionalidad para eliminar estudiantes, materias, notas o carreras del sistema
5. **Modularización adicional:** Crear módulos específicos para operaciones sobre cada lista (students.py, subjects.py, etc.). ✓ **(entrega 13/10)**
6. **Recursividad:** Aplicar recursividad en funciones que repiten estructuras similares, como los menús anidados o las búsquedas, reduciendo la redundancia de código y mejorando la claridad de las operaciones repetitivas.

Conclusiones

Muchas veces vemos aplicaciones diseñadas y terminadas que se ven muy bien, pero no nos ponemos a pensar cómo se logra ese resultado. Construir un programa es mucho más que escribir código, conlleva un proceso y trabajo necesario para poder lograr y obtener buenos resultados.

Desde el inicio de seleccionar una idea, mejorarla o perfeccionarla, elaborar los requisitos y funciones del programa hasta escribir el código, probarlo, y corregirlo si es necesario, conlleva bastante tiempo y recursos

Estamos muy contentos, con el resultado hasta ahora obtenido, nuestro programa no solo cumple todos los requerimientos planteados desde el principio sino que los supera ampliamente incorporando nuevas y mejores funciones.

"Sin requerimientos o diseño programar sólo es el arte de agregar errores a un archivo vacío" .- Louis Srygley

Bibliografía

UADE

- https://virtualbsp.uade.edu.ar/d2l/le/content/49250/viewContent/660248/View
- https://virtualbsp.uade.edu.ar/d2l/le/content/49250/viewContent/660258/View
- https://virtualbsp.uade.edu.ar/d2l/le/content/49250/viewContent/660266/View
- https://virtualbsp.uade.edu.ar/d2l/le/content/49250/viewContent/660292/View
- https://virtualbsp.uade.edu.ar/d2l/le/content/49250/viewContent/660285/View
-

Python Docs

- https://docs.python.org/3/library/stdtypes.html
- https://docs.python.org/3/tutorial/controlflow.html
- https://docs.python.org/es/3.8/tutorial/datastructures.html
- https://docs.python.org/3/library/csv.html
-